Durham University

Java Collections

Mladen Ivkovic

COMP2221 Programming Paradigms – Object-oriented Programming

Lecture 6 Recap

Join by Web

PollEv.com /mladenivkovic356

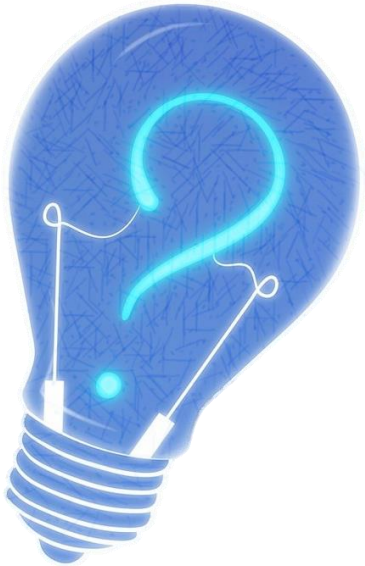
Durham University

Durham University

In This Lecture...

What Will You Learn in this Lecture?

- Java Generics
- Java Collections: `Lists`, `Iterators` & `HashMaps`
- Understanding `Arrays`, `Lists`, `LinkedList`, `Iterators`, `HashMaps` & `HashSets`



Durham University

Durham University

Java Generics

Java Generics

- Generics: **Parametrised Types**
  - i.e. lets us **use types as parameters**
  - Let us write code for different data/object types using a single class or method

- Use "diamond operator" `<T>` to denote generic type
  - Use `<T, U, V, …>` if you need more than 1 generic type in your class/method

Java Generics – Example: Without Generics

```
class Chocolate { // code... }
class Toy { // code... }

class BoxOfChocolates {
  Chocolate[] content;
  BoxOfChocolates(Chocolate[] content) {
    this.content = content;
  }
  void printContentSize() {
    System.out.println(this.content.length);
  }
}

class BoxOfToys {
  Toy[] content;
  BoxOfToys(Toy[] content){ this.content = content; }
  void printContentSize() {
    System.out.println(this.content.length);
  }
}
```

```
public class Generics {

  public static void main(String[] args) {
    Chocolate[] c = { new Chocolate(),
                      new Chocolate()};
    BoxOfChocolates bc = new BoxOfChocolates(c);
    bc.printContentSize();

    Toy[] t = {new Toy(), new Toy(), new Toy()};
    BoxOfToys bt = new BoxOfToys(t);
    bt.printContentSize();
  }
}
```

We have to implement a new class for each type of content we want to store in a "Box"!

Java Generics – Example: Using Generics

```
class Chocolate { // code... }
class Toy { // code... }

// generic class
class Box<T> {
  // from here on, use `T` instead of a specific type!
  T[] content;
  Box(T[] content){
    this.content = content;
  }
  void printContentSize() {
    System.out.println(this.content.length);
  }
}

// Don't let the syntax confuse you:
// I use `T[]` here because I want an array.
// To use a scalar variable, just use `T` instead.
```

```
public class Generics {

  public static void main(String[] args) {
    Chocolate[] c = {new Chocolate(),
                     new Chocolate()};
    Box<Chocolate> bc = new Box<Chocolate>(c);
    bc.printContentSize();

    Toy[] t = {new Toy(), new Toy(), new Toy()};
    Box<Toy> bt = new Box<Toy>(t);
    bt.printContentSize();
  }
}
```

Single implementation of **Box** for any type of content!

Durham University

Java Generics – Example for methods

```
public class Generics {

  public static <T> void printArray(T[] arr) {
    for (int i = 0; i < arr.length; i++) {
      System.out.print(arr[i] + ",");
    }
    System.out.println();
  }

  public static void main(String[] args) {

    Integer[] ia = {1,2,3,4};
    printArray(ia);

    Double[] da = {5., 6., 7.};
    printArray(da);
  }
}
```

Durham University

Java Generics – Internal workings

- **Primitive types** (`int, float, double`) cannot be type parameters.
  - Use "Boxed types" `Integer, Float, Double` etc instead
- **Type erasure**:
  - Essentially, the compiler replaces `T` with `Object`
  - (This is why primitive types don't work – they don't extend `Object`)
  - → you can't use any method that `Object` doesn't know!

```
class GenCalculator<T> {
  T add(T a, T b) {
    return a + b;
  }
}
```

Error:
The operator + is undefined for the argument types (T, T)

Bounded Generics

- You can **restrict** the type parameters:
  E.g. `<T extends Number>` instead of `<T>`
  - `Number` here is an example. You can use any class/interface instead.
  - This limits what types (and their extensions/subtypes) our class/method will accept
- But: Compiler will replace `T` with `Number`, not `Object`
  - → you can use any method that `Number` knows!

```
class GenCalculator<T extends Number> {
  double add(T a, T b) {
    return a.doubleValue() + b.doubleValue();
  }
}
```

This works!

Durham University

Java Collections

What is the Java Collections Framework (JCF)?

Definition: The **Java Collections Framework (JCF)** is a unified architecture for handling and manipulating groups of objects efficiently.

It provides a set of **interfaces and classes** for storing, retrieving, and processing data dynamically.

Key Benefits:

- **Reusability** → Standardized, tested components.
- **Performance** → Efficient algorithms for sorting, searching, and storing data.
- **Flexibility** → Supports dynamic resizing (unlike `Arrays`).
- **Thread Safety** → Some collections support concurrency (e.g., `ConcurrentHashMap`).
- **Reduced Boilerplate Code** → Eliminates the need for manual data structure implementation.

Why Java Collections Matter

- Java Collections **extend arrays** by providing **dynamic, efficient** data storage.
- Collections are **pre-built, reusable** solutions for managing objects in Java.
  - No need to re-invent the wheel!
- Collections make handling **large data sets** easier and more performant.
- **Key interfaces**: `List, Set, Map, Queue`

Why Use Collections Instead of Arrays?

Arrays (Fixed Size)

```
int[] numbers = new int[5];
```

Limitations:

- Fixed **size** (cannot grow/shrink dynamically).
- Need **explicit looping** for insertions and deletions.
- **No built-in methods** for sorting, searching, or managing data.

Collections (Dynamic & Flexible)

```
List<Integer> numbers = new ArrayList<>();
```

Advantages:

- **Dynamic Resizing**
- **More Built-in Methods** (`add()`, `remove()`, `contains()`, `sort()`)
- **Supports Different Data Types**

Durham University

Example: Collections provide a more powerful way to handle data compared to arrays

```
// using arrays (fixed size)
Int[] arr = new int[3];
Arr[0] = 10; arr[1] = 20; arr[2] = 30;
// arr[4] = 40; // Error: Index 3 out of bounds for length 3


// Using ArrayList (dynamic size)
List<Integer> list = new ArrayList<>();
List.add(10); list.add(20); list.add(30);
List.add(40); // No size limitation!
```

Java Collections Hierarchy

The Java Collections Framework is **divided into four main categories**:

1. **`List`:** Ordered collection ; allows duplicates.
2. **`Set`:** Unique elements, no duplicates.
3. **`Map`:** Key-value pairs, no duplicate keys.
4. **`Queue`:** FIFO structure, used in task scheduling.

Key Interfaces & Implementations

Interface	Description	Common Implementations
List	Ordered, allows duplicates	`ArrayList`, `LinkedList`
Set	Unique elements, no duplicates	`HashSet`, `TreeSet`
Map	Key-value pairs	`HashMap`, `TreeMap`, `LinkedHashMap`
Queue	FIFO, used in scheduling	`PriorityQueue`, `Deque`

Example of Different Implementations

```
List<String> list = new ArrayList<>();   // Dynamic list
Set<Integer> set = new HashSet<>();       // Unique elements
Map<String, Integer> map = new HashMap<>(); // Key-value storage
```

Arrays vs. Lists

- **Arrays**:
  - Fixed size, efficient for **indexed access**
- **Lists** (ArrayList, LinkedList):
  - Dynamic size, can be better for **performance** (trade-offs need to take place)**:**

Data Structure	Advantage	Caveat
Array	fast access by index	fixed size
ArrayList	dynamic resizing	slower insert/remove
LinkedList	fast insertion/removal of elements	Slow random access

ArrayList Example

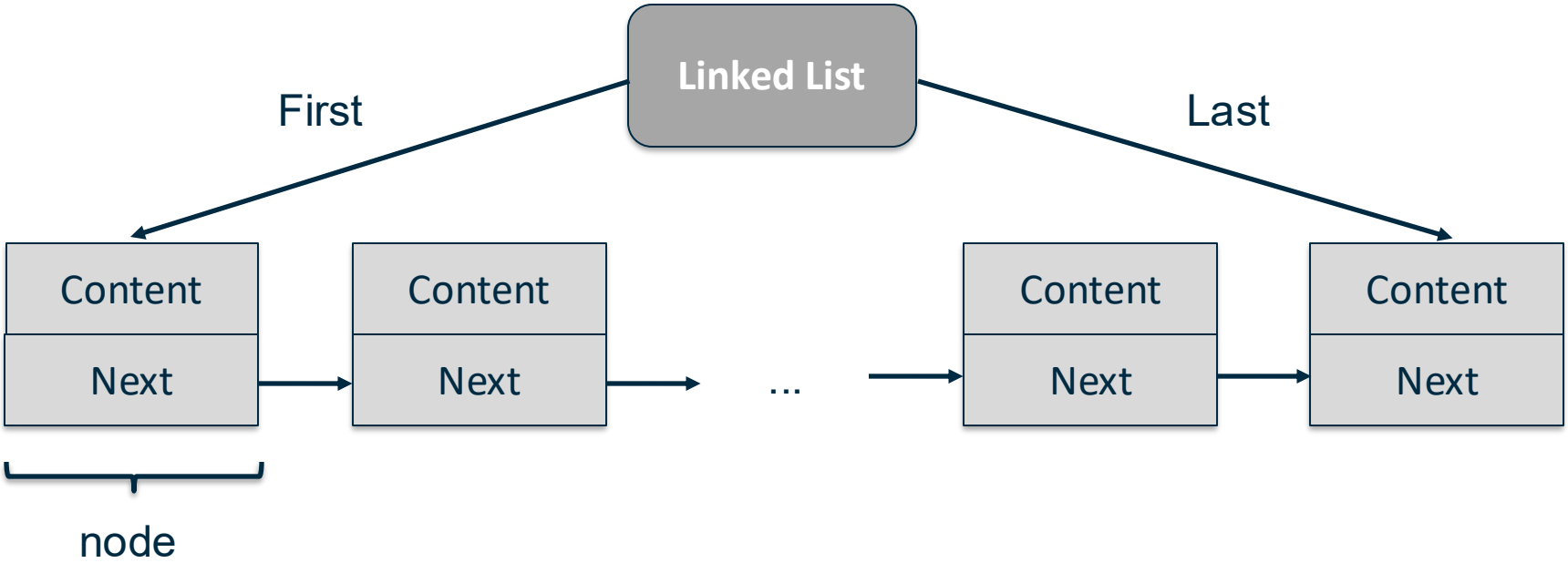
- Works like built-in fixed size array: Keeps elements in order
- Will resize if needed
- Use `get(int index)` to access specific elements

```java
import java.util.ArrayList;

public class Collections {

  public static void main(String[] args) {
    ArrayList<String> cars = new ArrayList<String>();
    cars.add("Porsche");
    cars.add("Ferrari");
    cars.add("Lamborghini");

    // Print entire array
    System.out.println(cars);

    // Access specific element
    System.out.println(cars.get(0));
  }
}
```

`LinkedList`

- Stores **elements in "containers" (nodes)**
- List has access to first (and last) node
- Every **node holds information** which node is next in list (*singly linked list*)
- *Doubly linked list*: Every node also knows which node is **previous** in list

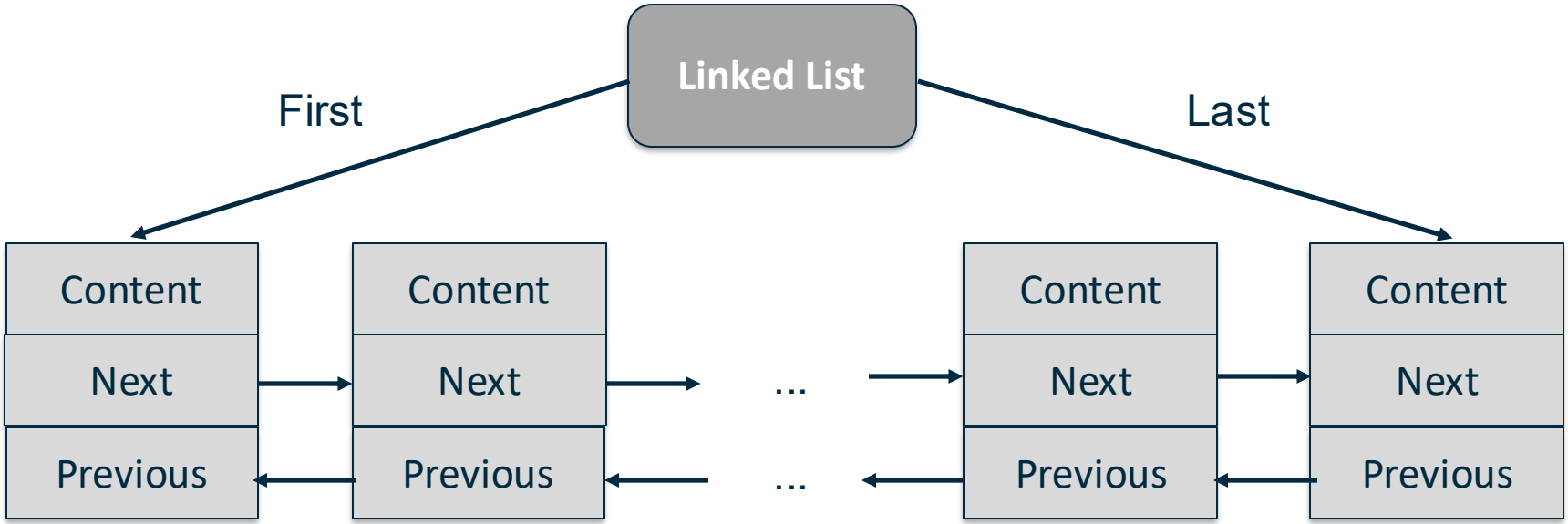
Singly Linked List



Durham University

Doubly Linked List

`java.util.LinkedList` is a doubly linked list



Durham University

Linked List

- **Slow random access of elements**:
  - Need to transverse all nodes/elements until you find the one you're looking for
  - In arrays: Index specifies where data is kept in memory
- **Quick insertion/removal**
  - Simply adapt references to previous/next element of 2 nodes
  - In arrays: Need to shift around memory to keep storage contiguous
- **Do**: Use LL when you need many insertions and removals.
- **Do not**: Use LL when you need frequent random access (via `get(index)`)
- **Do:** Use (fixed) arrays when you use fixed data size, frequent random access

`LinkedList` Example

- Works like `ArrayList`
- Use `` `get(int index)` `` to access specific elements

```
import java.util.LinkedList;

public class Collections {

  public static void main(String[] args) {
    LinkedList<String> cars = new LinkedList<String>();
    cars.add("Porsche");
    cars.add("Ferrari");
    cars.add("Lamborghini");

    // Print entire list
    System.out.println(cars);

    // Access specific element
    System.out.println(cars.get(0));
  }
}
```

Looping over lists

```
import java.util.LinkedList;
import java.util.List;

public class Collections {
  public static void main(String[] args) {
    List<String> cars =
                  new LinkedList<String>();

    cars.add("Porsche");
    cars.add("Ferrari");
    cars.add("Lamborghini");
```

```
    // for loop (not good for linked lists)
    for (int i = 0; i < cars.size(); i++) {
       System.out.println(cars.get(i));
    }

    // enhanced for loop / ForEach loop
    for (String listElement : cars) {
      System.out.println(listElement);
    }
  }
}
```

This works for `ArrayLists` as well

Looping over lists: `Iterators`

```java
import java.util.LinkedList;
import java.util.List;
import java.util.Iterator;

public class Collections {
  public static void main(String[] args) {
    List<String> cars =
                  new LinkedList<String>();

    cars.add("Porsche");
    cars.add("Ferrari");
    cars.add("Lamborghini");
    // use an iterator
    Iterator<String> it = cars.iterator();

    // hasNext(): returns true if next
    // element exists, false otherwise
    while(it.hasNext()){

      // next(): return the next element
      // in the iteration
      System.out.println(it.next());
    }
  }
}
```

This works for `ArrayLists` as well

Durham University

Looping over lists: `Iterators`

Why use an `Iterator`?

- **`Iterator` allows removal** (`iterator.remove()`)
- **For-loop does not** (`ConcurrentModificationException`)
- Applicable to **all collection objects** (`Lists`, `Sets`, `Maps`…)
- **`ListIterator`** allows backwards loops

Looping over lists: Removing elements

```
// unsafe removal, do not do this!
for (int i = 0; i < cars.size(); i++) {
  String c = cars.get(i);
  if (c.equals("Lamborghini")) {
    cars.remove("Lamborghini");
  }
}

// unsafe removal, do not do this!
for (String c: cars) {
  if (c.equals("Ferrari")) {
    cars.remove("Ferrari");
  }
}
```

```
Exception in thread "main"
java.util.ConcurrentModificationException
```

```
Iterator<String> it = cars.iterator();

while(it.hasNext()){
  String c = it.next();
  if (c.equals("Ferrari")) {
    // safe removal, do this!
    it.remove();
  }
}
```

Durham University

`HashMap` (Key-Value Pairs)

- **What is a `HashMap`?**
  - Stores data as **key-value pairs**
  - **Maps** keys to values
    - Compare arrays – arrays "map" index to values
    - If it helps to wrap your head around the concept, try thinking of maps as "arrays indexed by something which isn't an integer"
  - Keys must be **unique**
    - Compare arrays – you can't have index N store two different values
  - Uses **hashing** for fast lookups
  - No guaranteed order (unlike `List`)

HashMap Example

```
import java.util.HashMap;

public class Collections {
  public static void main(String[] args) {
    // HashMap<KeyType, ValueType>
    // It doesn't need to be <String, String>, it can be anything you like!
    HashMap<String, String> capitalCities = new HashMap<String, String>();

    // Add keys and values (Country, City)
    capitalCities.put("England", "London");
    capitalCities.put("Switzerland", "Bern!!");
    capitalCities.put("Switzerland", "Bern"); // Duplicate - will be overwritten
    capitalCities.put("Germany", "Berlin");

    System.out.println(capitalCities);
    System.out.println(capitalCities.get("Germany"));
  }
}
```

Output:

```
{England=London, Switzerland=Bern, Germany=Berlin}
Berlin
```

`HashSet` (Unique Data Storage)

What is a HashSet?

- A `Set` implementation that **stores only unique values**
- Uses **`HashMap` internally**
- Does **not** preserve order

When to use?

- Removing **duplicates from a list**
- Checking **unique usernames**

HashSet Example

```
import java.util.HashSet;
public class Collections {
  public static void main(String[] args) {

    HashSet<String> fruit = new HashSet<String>();

    // Add unique elements
    fruit.add("Apple");
    fruit.add("Banana");
    fruit.add("Banana");  // duplicate - will be
                          // overwritten
    fruit.add("Clementine");
    System.out.println(fruit);

    // does element exist?
    if (fruit.contains("Apple")) {
        System.out.println(
            "Apple is in HashSet.");
    }

  }
}
```

Output:

```
[Apple, Clementine, Banana]
Apple is in HashSet.
```

Durham University

Performance Comparisons

Operation	Array	ArrayList	LinkedList	HashMap
Access (get)	O(1) ✅	O(1) ✅	O(n) ❌	O(1) ✅
Insert (add)	O(n) ❌	O(1) ✅	O(1) ✅	O(1) ✅
Delete (remove)	O(n) ❌	O(n) ❌	O(1) ✅	O(1) ✅

- **ArrayList → Best for fast lookups, bad for insert/remove**
- **LinkedList → Best for frequent insertions/removals**
- **HashMap → Best for key-value mapping & fast lookups**
- **HashSet → Best for unique element storage**
- **Avoid using LinkedList for large lists unless frequent modifications are needed**

Java Collections Framework

- The JCF offers more collections than we can cover
- Familiarise yourself with them – they can be very useful!
- https://docs.oracle.com/javase/tutorial/collections/implementations/index.html

General-purpose Implementations

Interfaces	Hash table Implementations	Resizable array Implementations	Tree Implementations	Linked list Implementations	Hash table + Linked list Implementations
Set	HashSet		TreeSet		LinkedHashSet
List		ArrayList		LinkedList	
Queue					
Deque		ArrayDeque		LinkedList	
Map	HashMap		TreeMap		LinkedHashMap

Summary

- **Collections improve flexibility** compared to arrays
- **`ArrayList` vs `LinkedList`**: Choose based on **performance needs**
- **`HashMap` enables fast key-value lookups**
- **`HashSet` ensures uniqueness**
- **Use `Iterators` for safe list traversal**

Durham University

The End

Any Questions?